U·tad

Unidad 3 — React Core

Web Development I, the Client · 3.º de Ingeniería del Software · 2026–2027

React 19.2 con Vite · JSX y componentes · Props y estado · Eventos y efectos · Hooks esenciales · Cuaderno de ejercicios 3 · Actividad A1 · Misión M3 «Reactor Core»

Al terminar esta unidad sabrás…

- Explicar qué hace React distinto del DOM «a mano»: **declarativo**, reconciliación, «tú cambias el estado, no la pantalla».
- Crear un proyecto con **Vite** y escribir **JSX** sin tropezar con `className`, las llaves y el nodo raíz.
- **Descomponer** una interfaz en componentes pequeños con **props** claras y `children`.
- Gestionar el estado con `useState` sin mutar nada y **derivando** en vez de duplicar.
- Pintar **listas con `key` estable** y condicionales legibles (sin el `0` fantasma).
- Usar `useEffect` **solo cuando toca**, con dependencias correctas y **limpieza**.
- **Levantar el estado** al ancestro común y compartir datos con `useContext`.
- Construir un juego completo gobernado por `useReducer`: **Memoria Cósmica**.

Mapa de la unidad

1. ¿Qué es React? Declarativo y DOM virtual
2. Puesta en marcha con Vite
3. JSX
4. Componentes y props
5. Renderizado condicional y listas
6. Estado con `useState`
7. Eventos
8. Levantar el estado y flujo unidireccional
9. `useEffect`
10. Otros hooks, hooks personalizados y React 19
11. Pensar en React y buenas prácticas
12. **Actividad A1:** code review al replicante
13. **Reto jefe:** Memoria Cósmica
14. **Arena:** misión M3 y defensa oral

> Cuaderno 3 intercalado: 4 calentamientos 🔴, 3 misiones 🟡, 1 reto jefe 🟢.

1 · ¿Qué es React?

Una **librería** para construir interfaces: describes **qué** se ve para un estado dado y React se ocupa del **cómo**.

Imperativo frente a declarativo

```
// Imperativo (U1): tú manipulas el DOM paso a pa
const titulo = document.createElement("h1");
titulo.textContent = "Hola";
document.body.appendChild(titulo);

// …y cuando cambia algo, vuelves a dar órdenes:
titulo.textContent = "Adiós";
```

```
// Declarativo (React): describes el resultado
function Saludo({ despedida }) {
  return <h1>{despedida ? "Adiós" : "Hola"}</h1>;
}
```

- Tú no tocas la pantalla: **cambias el estado** y React recalcula qué se ve.
- Interfaz **basada en componentes**: piezas reutilizables que se combinan como un Lego.

> React es una **librería**, no un framework: no impone carpetas, ni router, ni forma de pedir datos.

El DOM virtual y por qué se usa con un framework

Reconciliación (idea general)

1. Tocar el DOM real es **caro** (U1: *layout* y *paint*).
2. React guarda en memoria una representación ligera: el **DOM virtual**.
3. Al cambiar el estado genera una versión nueva, la **compara** con la anterior…
4. …y aplica al DOM real **solo la diferencia**.

> No hace falta dominar los detalles internos: basta con saber que React actualiza el DOM de forma eficiente **por ti**.

¿Y el framework?

- React solo no resuelve el **enrutado**, los datos en el servidor ni el **SEO**.
- Por eso en proyectos reales va dentro de un framework: **Next.js** (Unidad 6). La propia documentación lo recomienda.
- En esta unidad usamos una **SPA** sencilla creada con **Vite** para centrarnos en los fundamentos.
- Siempre **componentes como funciones**: las clases son legado.

2 · Puesta en marcha

Vite en un minuto, la estructura del proyecto y el punto de entrada.

Un proyecto React con Vite

```
npm create vite@latest mi-app -- --template rea
cd mi-app
npm install
npm run dev     # http://localhost:5173
```

> ⚠️ **Create React App está descontinuado.**
> No lo uses aunque lo veas en tutoriales o te lo sugiera la IA. Hoy: Vite para una SPA, Next.js para aplicaciones completas.

```
mi-app/
├── index.html          # punto de entrada HTML
├── package.json        # dependencias y scripts
├── vite.config.js      # configuración de Vite
└── src/
    ├── main.jsx        # arranca React y monta la
    ├── App.jsx         # componente raíz
    └── components/     # tus componentes (conven
```

- Un **único proyecto** para todo el cuaderno: cada ejercicio en su carpeta dentro de `src/`.

`main.jsx` : conectar React con el HTML

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).ren
  <StrictMode>
    <App />
  </StrictMode>
);
```

- `createRoot(...).render(...)`: la **única** vez que tocas el DOM a mano en toda la aplicación.
- `StrictMode` no afecta a producción; en desarrollo activa comprobaciones extra: ejecuta dos veces ciertas funciones para que descubras efectos mal escritos. **No lo quites.**
- Instala **React Developer Tools** (Chrome, Firefox, Edge): pestañas *Components* y *Profiler* para ver el árbol, las props y el estado en tiempo real.

3 · JSX

Parece HTML, pero es JavaScript: expresiones entre llaves, atributos en *camelCase*, un solo nodo raíz.

JSX: expresiones entre llaves y diferencias con HTML

```
const nombre = "Ada";
const elemento = (
  <p className="saludo">
    Bienvenida, {nombre}. Hoy es {new Date().ge
  </p>
);
```

- JSX es **azúcar sintáctico**: se transforma en llamadas a funciones de React. No es HTML ni un string.
- Dentro de `{}` van **expresiones** (algo que produce un valor): un ternario o un `.map()`, sí; un `if` o un `for`, no.

HTML	JSX	Motivo
`class`	`className`	palabra reservada en JS
`for`	`htmlFor`	palabra reservada en JS
`onclick`	`onClick`	eventos en *camelCase*
`tabindex`	`tabIndex`	atributos en *camelCase*

- Todas las etiquetas se **cierran**: `<img />`, `<br />`, `<input />`.

Un solo nodo raíz: Fragments y comentarios

```jsx
function Datos() {
  return (
    <>
      <h2>Título</h2>
      <p>Párrafo asociado.</p>
    </>
  );
}
```

Un componente devuelve **un único nodo raíz**. El **Fragment** `<>…</>` agrupa hermanos **sin añadir un** `<div>` al DOM.

```jsx
function Ejemplo() {
  return (
    <div>
      {/* Esto es un comentario en JSX */}
      <p>Contenido</p>
    </div>
  );
}
```

Los comentarios van como **expresiones** de JavaScript: `{/* … */}`. Un `<!-- -->` de HTML no funciona.

4 · Componentes y props

Un componente es una **función que devuelve JSX**; las props son sus **argumentos**.

Componentes: funciones con mayúscula

```jsx
function Boton() {
  return <button>Pulsa aquí</button>;
}

function Tarjeta() {
  return (
    <article className="tarjeta">
      <h3>Producto</h3>
      <Boton />        {/* composición */}
    </article>
  );
}
```

- **PascalCase** no es solo estilo: `<button>` es una etiqueta del DOM; `<Boton>` es **tu componente**. Es una **regla** de JSX.

```jsx
// Boton.jsx
export default function Boton() {
  return <button>Pulsa aquí</button>;
}
```

```jsx
// App.jsx
import Boton from "./Boton.jsx";

export default function App() {
  return <Boton />;
}
```

- Un componente por fichero, exportado por defecto e importado donde se usa (módulos ES de la U2).
- Los **componentes de clase** son la forma antigua: reconócelos, no los escribas.

Props: los argumentos del componente

```jsx
function Saludo(props) {
  return <p>Hola, {props.nombre}</p>;
}

// Uso desde el padre: atributos en el JSX
<Saludo nombre="Ada" />
```

Las props son los datos que un **padre** pasa a un **hijo**. El hijo las recibe como **un objeto**.

```jsx
// Lo habitual: desestructurar en los parámetros
function Saludo({ nombre }) {
  return <p>Hola, {nombre}</p>;
}

// Valores por defecto: sintaxis de JS normal
function Boton({ texto = "Aceptar" }) {
  return <button>{texto}</button>;
}
```

> Son **parámetros de función**: desestructuración y valores por defecto son los de la U2, nada nuevo.

`children` y la regla de oro: las props son de solo lectura

```jsx
function Panel({ titulo, children }) {
  return (
    <section className="panel">
      <h2>{titulo}</h2>
      {children}
    </section>
  );
}

// Uso: lo que va ENTRE las etiquetas llega como 
<Panel titulo="Avisos">
  <p>Este párrafo llega como children.</p>
</Panel>
```

- `children` es una prop especial: el contenido escrito entre `<Panel>` y `</Panel>`. Permite componentes **envoltorio** (paneles, modales, *layouts*).
- **Un componente nunca modifica sus props.** Fluyen del padre al hijo y son **inmutables** desde el hijo.
- Si un valor tiene que **cambiar con el tiempo**, no es una prop: es **estado** (siguiente bloque).

> ⚠️ `props.nombre = "otro"` rompe el flujo de datos de React. Nunca.

Ahora tú · 1.1 La tripulación

⏱ **10–15 min** · 🔴 **sin IA** · cuaderno 3, nivel 1

Estás fichando a la tripulación de una nave.

1. Componente `FichaTripulante` con props `nombre`, `rol` y `especie` (por defecto `"humana"`).
2. Devuelve un `<article>` con un título y dos líneas de texto.
3. Desde `App`, renderiza al menos **cuatro** tripulantes, alguno **sin** pasar `especie`.

Resultado esperado: cuatro fichas en pantalla; la que no recibe `especie` muestra «humana».

```
function FichaTripulante({ nombre, rol, especie
  // …
}
```

> Pista: desestructura en los parámetros y usa el valor por defecto de JavaScript.

5 · Renderizado condicional y listas

`if`, ternario y `&&` para decidir qué se ve; `.map()` y `key` para pintar colecciones.

Tres patrones de renderizado condicional

```jsx
// 1. if antes del return: estructuras muy distintas
function Saludo({ usuario }) {
  if (!usuario) {
    return <p>Inicia sesión, por favor.</p>;
  }
  return <p>Hola, {usuario.nombre}</p>;
}

// 2. Ternario: elegir entre dos opciones dentro del J
function Estado({ activo }) {
  return <span>{activo ? "En línea" : "Desconectado"}<
}
```

```jsx
// 3. &&: mostrar algo solo si la condición es cierta
function Bandeja({ mensajes }) {
  return (
    <div>
      {mensajes.length > 0 && (
        <p>Tienes {mensajes.length} mensajes nuevos.</
      )}
    </div>
  );
}
```

> ⚠️ `{mensajes.length && <p>…</p>}` pinta un `0` en pantalla si la lista está vacía. Usa siempre una comparación que devuelva un **booleano**.

DEMO EN VIVO

El `0` fantasma (simulación en JS puro)

Esto **no es React**: es lo que evalúa JavaScript dentro de `{avisos && <p>…</p>}`. React pinta números; no pinta `false`.

```
const avisos = 0;

console.log("avisos && 'párrafo'     →", avisos && "párrafo");      // 0     → React pinta "0"
console.log("avisos > 0 && 'párrafo' →", avisos > 0 && "párrafo");  // false → React no pinta nada

const conAvisos = 2;
console.log("con 2 avisos            →", conAvisos > 0 && "párrafo"); // "párrafo"

// Prueba: ¿qué devuelve  "" && "párrafo" ?  ¿Y  null && "párrafo" ?  ¿Los pintaría React?
```

Listas con `.map()` y la prop `key`

```jsx
function ListaTareas({ tareas }) {
  return (
    <ul>
      {tareas.map((tarea) => (
        <li key={tarea.id}>{tarea.texto}</li>
      ))}
    </ul>
  );
}
```

- `.map()` de la U2: transforma datos en elementos. La `key` va en el elemento que **devuelve** el `map`.

¿Para qué sirve `key`?

- Identifica cada elemento **entre renders**, no dentro del render: así React sabe qué ha cambiado, entrado o salido al reconciliar.
- Debe ser **única entre hermanos** y **estable**: un `id` del propio dato.

> ⚠️ **No uses el índice del array** si la lista puede reordenarse, filtrarse o crecer por el medio. Al borrar en medio, todas las keys posteriores se reasignan y React reutiliza el componente equivocado: inputs con el valor de otra fila, foco perdido.

Ahora tú · 1.2 Estado de la esclusa y 1.3 Ranking de la sala arcade

⏱ **2 × 10 min** · 🔴 **sin IA** · cuaderno 3, nivel 1

1.2 Componente `Esclusa` con props `abierta` (booleana) y `avisos` (número):

- «🟢 Esclusa abierta» / «🔴 Esclusa cerrada» con un **ternario**.
- «⚠️ Tienes N avisos pendientes» **solo** si hay avisos, con `&&`.
- Renderiza tres: abierta con 2, cerrada con 0, abierta con 0.

Esperado: sin avisos no aparece el párrafo… **ni un `0` suelto**.

1.3 Componente `Ranking` que recibe el array y pinta un `<ol>` con «JUGADOR — puntos»; medalla 🏅 si supera 8000.

```
const RANKING = [
  { id: "a1", jugador: "NOVA", puntos: 9800 },
  { id: "b2", jugador: "PIXEL", puntos: 8650 },
  { id: "c3", jugador: "KIRA", puntos: 7400 },
  { id: "d4", jugador: "BYTE", puntos: 5200 },
];
```

Esperado: medalla en los dos primeros y **cero warnings de `key`** en consola.

6 · Estado con `useState`

La **memoria** del componente: cambia con el tiempo y, al cambiar, React vuelve a pintar.

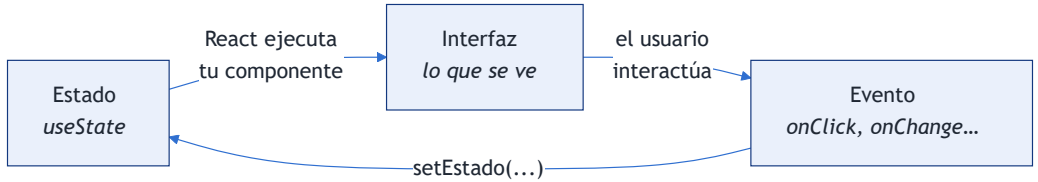
`useState`: el ciclo entero de React

```
import { useState } from "react";

function Contador() {
  const [cuenta, setCuenta] = useState(0);

  return (
    <button onClick={() => setCuenta(cuenta + 1)}>
      Has pulsado {cuenta} veces
    </button>
  );
}
```

- `useState(inicial)` devuelve **[valor actual, función para cambiarlo]**.
- Una variable normal no sirve: React no se entera y su valor se perdería en cada render.



> Tú no tocas la pantalla: **cambias el estado**. Cuando algo «no se actualiza», la pregunta es siempre la misma: ¿de verdad ha cambiado el estado?

El estado es una instantánea

```
function Ejemplo() {
  const [cuenta, setCuenta] = useState(0);

  function manejar() {
    setCuenta(cuenta + 1);
    setCuenta(cuenta + 1);
    setCuenta(cuenta + 1);
  }

  return <button onClick={manejar}>Cuenta: {cuen
}
```

- Tres llamadas… y la cuenta sube **1**, no 3.
- En ese render `cuenta` vale `0`: las tres líneas son `setCuenta(0 + 1)`.
- Dentro de un render, el valor del estado es **fijo**: una foto tomada al renderizar. El valor nuevo llega en el **siguiente** render.
- No es magia: es una variable capturada por un **closure** (U2) que no cambia mientras dura el render.

> Es la idea que más cuesta interiorizar de toda la unidad. La demo siguiente la desmonta.

DEMO EN VIVO

Simulación de `useState` en JS puro (esto no es React)

```
// Simulación mínima de useState: la cola de actualizaciones y el valor del render
let valorReal = 0;
const cola = [];

function render() {
  const cuenta = valorReal;                 // ← la INSTANTÁNEA de este render
  const setCuenta = (v) => cola.push(v);    // encola, no cambia 'cuenta' aquí

  // --- esto es lo que escribes tú en el componente ---
  setCuenta(cuenta + 1);
  setCuenta(cuenta + 1);
  setCuenta(cuenta + 1);
  // ---------------------------------------------------

  console.log("Durante el render, cuenta vale:", cuenta);
  console.log("Se ha encolado:", cola.slice());
```

La función actualizadora y el *batching*

```
function manejar() {
  setCuenta((c) => c + 1);
  setCuenta((c) => c + 1);
  setCuenta((c) => c + 1);
}
// Ahora sí sube de 3 en 3
```

- Pasa una **función** a la actualizadora cuando el nuevo valor dependa del **anterior real**: React la aplica sobre el valor más reciente de la cola.
- Regla práctica: `set(x)` para valores nuevos; `set(prev => …)` para incrementos, *toggles* y cambios encadenados.

Actualizaciones por lotes (*batching*)

- React **agrupa** todas las actualizaciones del mismo manejador y hace **un solo re-render** al final.
- Desde React 18 también en código **asíncrono** (promesas, `setTimeout`, eventos nativos).
- Por eso el estado no cambia «en mitad» de tu función: cambia para el **siguiente** render.

> Ejercicio 3 de los apuntes: botón «+3» que llama tres veces a `setCuenta` y un comentario explicando por qué `cuenta + 1` no funcionaría.

DEMO EN VIVO

Inmutabilidad: React compara referencias, no contenido

```
const usuario = { nombre: "Ana", edad: 29 };

// ❌ Mutar: cambias el contenido, pero la referencia es la misma
const mutado = usuario;
mutado.edad = 30;
console.log("¿React ve un cambio?", !Object.is(usuario, mutado)); // false → NO repinta

// ✅ Copiar: objeto nuevo, referencia nueva
const copiado = { ...usuario, edad: 31 };
console.log("¿React ve un cambio?", !Object.is(usuario, copiado)); // true → repinta

// Lo mismo con arrays: push muta, los métodos que devuelven copia no
const tareas = [{ id: 1, hecha: false }];
const empujado = tareas; empujado.push({ id: 2, hecha: false });
const concatenado = [...tareas, { id: 3, hecha: false }];
console.log("push →", !Object.is(tareas, empujado));       // false
```

Patrones inmutables con objetos y arrays

```
// ❌ MAL: mutar directamente
usuario.edad = 30;
setUsuario(usuario);

// ✅ BIEN: objeto nuevo con spread
setUsuario({ ...usuario, edad: 30 });
```

Mutan (evitar)	Devuelven copia (usar)
`push`, `pop`, `splice`	`[...arr, x]`, `filter`, `map`
`sort`, `reverse`	`toSorted`, `toReversed`
`obj.prop = v`	`{ ...obj, prop: v }`

```
// Añadir a un array sin mutar
setTareas([...tareas, nuevaTarea]);

// Eliminar sin mutar
setTareas(tareas.filter((t) => t.id !== id));

// Modificar un elemento sin mutar
setTareas(
  tareas.map((t) => (t.id === id ? { ...t, hecha: true
);
```

> Las tres operaciones de todas las misiones del cuaderno: añadir, eliminar, modificar. Logro 🧊 **Inmutable**: todo el cuaderno sin una sola mutación.

Ahora tú · 1.4 El aplausómetro

⏱ **15 min** · 🔴 **sin IA** · cuaderno 3, nivel 1

Componente `Aplausometro` con:

- Botón «👏 **Aplaudir**» que incrementa un contador.
- Botón **«Reiniciar»** que lo devuelve a 0.
- Un mensaje según el nivel: menos de 5 → «Tímido...», de 5 a 14 → «¡Se anima la sala!», 15 o más → «¡OVACIÓN TOTAL! 🎉 ».

Resultado esperado: el contador sube de uno en uno, el mensaje cambia solo al cruzar los umbrales y «Reiniciar» vuelve al estado inicial.

> 💡 El mensaje **no es estado**: derívalo del contador durante el render (un `if` antes del `return` o un ternario encadenado).

Pregunta trampa: ¿cuántos `useState` necesitas? Si has escrito dos, uno sobra.

7 · Eventos

`onClick`, `onChange`, `onSubmit`... reciben una **función**, no una llamada.

Eventos en React

```
function Boton() {
  function manejarClick() {
    alert("¡Clic!");
  }
  return <button onClick={manejarClick}>Pulsa</butt
}

function Formulario() {
  function manejarEnvio(evento) {
    evento.preventDefault(); // no recargar la pági
    console.log("Formulario enviado");
  }
  return <form onSubmit={manejarEnvio}>{/* … */}</fo
}
```

- Atributos en *camelCase* con una **función manejadora**. Sin `addEventListener`: el JSX lo declara.
- ⚠️ Se pasa la función **sin paréntesis**: `onClick={manejarClick()}` la **ejecuta durante el render** (y si llama a un `set`, bucle infinito).
- El **evento sintético** envuelve al nativo: igual en todos los navegadores, con `preventDefault()` y `stopPropagation()`.

```
// Pasar argumentos: envuelve la llamada en una flec
<li key={item.id} onClick={() => borrar(item.id)}>
```

Misión 1 · Trivial de la Academia 🚀

🟡 **IA como pair · ⏱ 90–120 min** · cuaderno 3, nivel 2

- 🎯 **Objetivo:** un quiz completo con estado, puntuación y pantalla de resultados.
- 🧰 **Practicas:** varios `useState`, «pantallas» por renderizado condicional, listas con `key`, estado derivado, inmutabilidad.

```
// src/misiones/quiz/preguntas.js
export const PREGUNTAS = [
  { id: "q1",
    texto: "¿Qué hook añade memoria a un componente?",
    opciones: ["useEffect", "useState", "useRef", "useCo
    correcta: 1 },
  // … q2, q3, q4
];
```

Fases (checkpoints)

1. **Pantalla de pregunta:** `Quiz.jsx` con `indice` y `puntos`; texto y opciones como botones (lista con `key`).
2. **Responder y avanzar:** acierto → +1; siempre → siguiente pregunta.
3. **Pantalla final:** si `indice` llega al final, componente `Resultado` («N de M aciertos», mensaje por porcentaje, «Jugar otra vez» que reinicia ambos estados).
4. **Feedback visual:** estado `seleccion` para marcar la opción (verde/roja) antes de avanzar.

Misión 1 · Criterios de aceptación y retos

✅ No está terminada hasta marcar todo:

- ☐ El quiz avanza pregunta a pregunta y la puntuación es correcta.
- ☐ La pantalla final muestra los aciertos y permite reiniciar.
- ☐ Los botones de opciones se generan con `.map()` y `key` estable.
- ☐ El porcentaje/mensaje final se **deriva** de `puntos`, sin guardarse en estado.
- ☐ No hay warnings en la consola del navegador.

💡 Pistas

- En la fase 4, guarda en `seleccion` el índice pulsado (o `null`) y calcula la clase CSS de cada botón **durante el render**.

🚀 Retos extra

- Barajar las preguntas al empezar: una **copia** desordenada, sin mutar `PREGUNTAS`.
- Barra de progreso («Pregunta 2 de 4») y racha de aciertos seguidos.

> Semáforo 🟡: pídele a la IA críticas y pistas («¿por qué falla esto?»), no la solución entera. Logro 🧠 **Cerebro de quiz**.

8 · Levantar el estado y flujo unidireccional

Los datos **bajan** por props; los cambios **suben** como llamadas a funciones. Los hermanos no se hablan.

Levantar el estado (*lifting state up*)

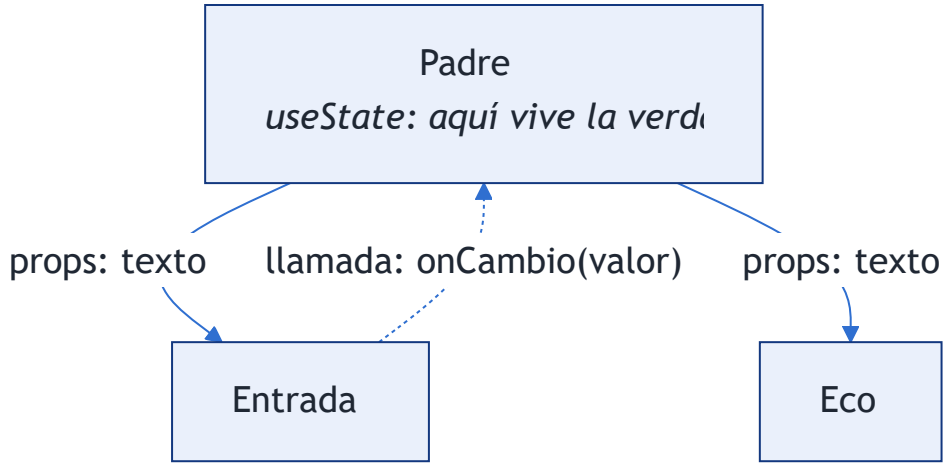
```jsx
function Padre() {
  const [texto, setTexto] = useState("");
  return (
    <>
      <Entrada texto={texto} onCambio={setTexto} />
      <Eco texto={texto} />
    </>
  );
}

function Entrada({ texto, onCambio }) {
  return <input value={texto} onChange={(e) => onCambio(e.targ
}

function Eco({ texto }) {
  return <p>Escribiste: {texto}</p>;
}
```

- ¿Dos hermanos necesitan el **mismo** dato? El estado sube al **ancestro común más cercano**.
- El padre lo pasa como **prop** y entrega también la **función** para modificarlo. Es la **única fuente de verdad**.
- `value` + `onChange` = **input controlado**: el valor lo dicta el estado de React (a fondo en la U4).

Flujo unidireccional



- Flechas continuas: los datos **bajan** por props.
- Flecha de puntos: los cambios **suben** como llamadas a funciones.
- **No hay flecha entre** `Entrada` **y** `Eco`: los hermanos nunca se hablan directamente; se coordinan a través del padre.
- Resultado: aplicaciones **predecibles**. Para depurar, sigue las flechas hacia arriba.

> Cuando el árbol crece y una prop tiene que atravesar cinco niveles solo para llegar abajo, aparece el *prop drilling*: lo resolvemos con `useContext` en el bloque 10.

Misión 2 · Forja de hábitos 🔥

🟡 **IA como pair · ⏱ 90–120 min** · cuaderno 3, nivel 2

- 🎯 **Objetivo:** gestor de hábitos donde el estado vive en el padre y los hijos solo reciben datos y funciones.
- 🧰 **Practicas:** levantar el estado, flujo unidireccional, inmutabilidad, input controlado.

```
AppHabitos            ← dueña del estado (array de hábitos
├── FormularioHabito  ← input controlado + «Añadir»
├── ListaHabitos      ← recibe hábitos y funciones por pro
│   └── TarjetaHabito ← «Hecho hoy» y «Eliminar»
└── PanelResumen      ← totales DERIVADOS
```

Fases: 1) versión **estática** con props · 2) estado arriba y «Hecho hoy» con `map` + spread · 3) añadir (`crypto.randomUUID()`) y eliminar (`filter`) · 4) resumen **derivado** («X de Y hoy», mejor racha).

✅ Criterios

- ☐ El array vive en un único `useState` en el padre.
- ☐ Los hijos modifican el estado solo vía funciones recibidas por props.
- ☐ Ninguna operación muta: solo spread, `map` y `filter`.
- ☐ El formulario limpia el input tras añadir y rechaza hábitos vacíos.
- ☐ `PanelResumen` tiene **cero** `useState`.

🚀 **Retos:** botón «Nuevo día» (resetea `hechoHoy` y rompe la racha de los no completados) · mini-kanban «Pendientes / Completados» con dos `filter` sobre el **mismo** estado.

9 · `useEffect`

Sincronizar el componente con lo que vive **fuera** de React: temporizadores, suscripciones, red, `localStorage`.

`useEffect`: un reloj con limpieza

```
import { useState, useEffect } from "react";

function Reloj() {
  const [hora, setHora] = useState(new Date());

  useEffect(() => {
    const id = setInterval(() => setHora(new Date
    // Limpieza: al desmontar o antes de re-ejecu
    return () => clearInterval(id);
  }, []); // [] → solo una vez, al montar

  return <p>{hora.toLocaleTimeString()}</p>;
}
```

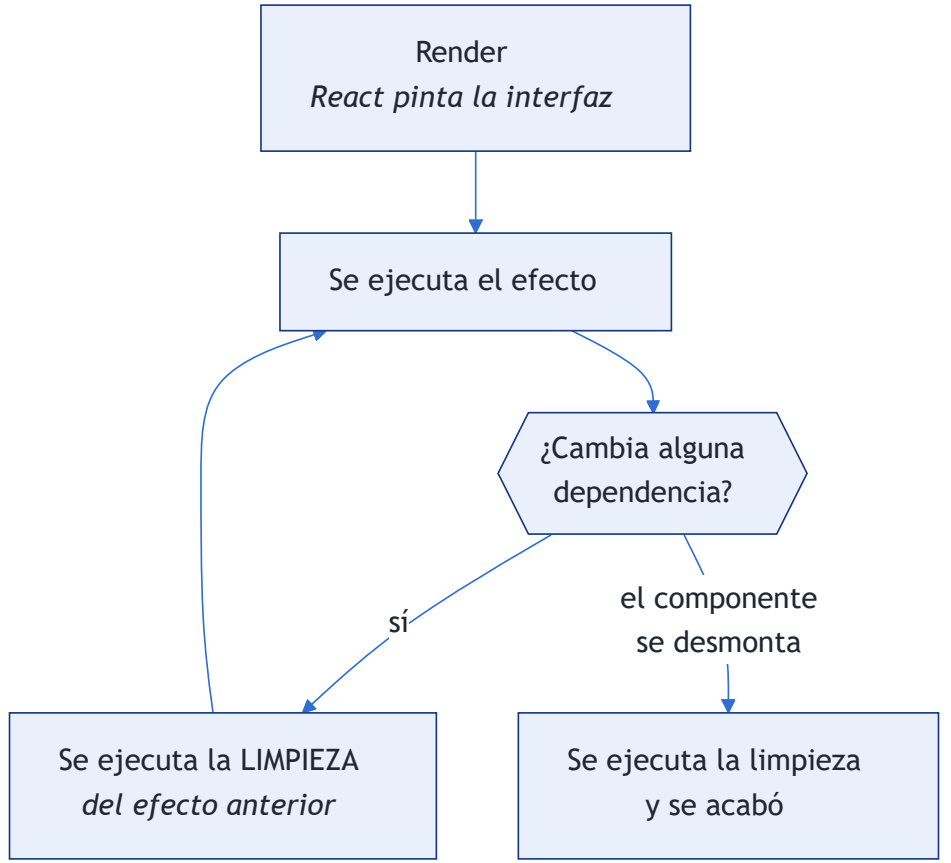
- El efecto corre **después** de que React haya pintado.
- Devuelve una **función de limpieza** si creaste algo que hay que deshacer.

> ⚠️ Con `StrictMode`, en **desarrollo** React monta → limpia → remonta el efecto: lo verás ejecutarse **dos veces** en consola. Es intencionado (prueba tu limpieza) y **solo en desarrollo**. Si el doble montaje rompe algo, el bug es tuyo, no de React.

Dependencias y limpieza

Segundo argumento	Cuándo se ejecuta
(ninguno)	Después de **cada** render
`[]`	Solo una vez, al **montar**
`[a, b]`	Al montar y cada vez que `a` o `b` cambian

- La limpieza corre al **desmontar** y **antes de cada nueva ejecución** del efecto.
- Sin limpieza, cada cambio de dependencia deja un temporizador más corriendo por detrás.



Cuándo NO usar un efecto

```
// ❌ Innecesario: un efecto para derivar un valor
const [nombreCompleto, setNombreCompleto] = useState("
useEffect(() => {
  setNombreCompleto(nombre + " " + apellido);
}, [nombre, apellido]);

// ✅ Calcúlalo directamente durante el render
const nombreCompleto = nombre + " " + apellido;
```

Guía oficial: **«You Might Not Need an Effect»** (react.dev). Muchos efectos sobran:

- **No** para **transformar datos para el render**: si sale de props o estado, calcúlalo en el render (o `useMemo` si es caro).
- **No** para responder a una **interacción**: eso va en el **manejador del evento**.
- **Sí** para sincronizar con algo **externo**: red, temporizadores, suscripciones, `document.title`, `localStorage`.

> Señal de alarma: un `useEffect` cuyo cuerpo es solo un `setEstado` a partir de otro estado. Es un valor derivado disfrazado, y encima provoca un render de más.

Reglas de los hooks

Los hooks (toda función que empieza por `use`) tienen **dos reglas innegociables**:

1. Solo se llaman en el **nivel superior** del componente: nunca dentro de bucles, condicionales o funciones anidadas. React necesita el **mismo orden** en cada render.
2. Solo se llaman desde **componentes de React** o desde **otros hooks** personalizados.

```
// ❌ Hook dentro de un if: el orden cambia entre r
if (mostrar) {
  const [x, setX] = useState(0);
}

// ✅ Siempre en el nivel superior; la condición, d
const [x, setX] = useState(0);
if (mostrar) { /* usa x */ }
```

> 💡 `eslint-plugin-react-hooks` (viene con la plantilla de Vite) verifica las reglas y avisa de **dependencias que faltan** en los efectos. **No lo silencies**: si protesta, pregúntate si ese código debería estar en un efecto.

10 · Otros hooks, hooks personalizados y React 19

`useRef` · `useContext` · `useReducer` · `useMemo` y `useCallback` · tus propios hooks · `use` y las Actions.

`useRef`: una caja que no provoca renders

```
import { useRef } from "react";

function CampoTexto() {
  const inputRef = useRef(null);
  return (
    <>
      <input ref={inputRef} />
      <button onClick={() => inputRef.current.focu
        Enfocar
      </button>
    </>
  );
}
```

`useRef` crea una «caja» mutable (`.current`) que **persiste entre renders** y cuyo cambio **no repinta**. Dos usos:

1. **Acceder a un nodo del DOM**: foco, medir, *scroll*, reproducir un vídeo.
2. **Guardar un valor mutable** que no debe disparar render: el `id` de un intervalo, el valor anterior de una prop.

> 📌 React 19: `ref` se pasa como una **prop normal** a tus componentes de función. En la mayoría de casos ya **no necesitas** `forwardRef`.

`useContext`: adiós al *prop drilling*

```
import { createContext, useContext } from "react";

const TemaContext = createContext("claro");

function App() {
  return (
    <TemaContext value="oscuro">
      <Barra />
    </TemaContext>
  );
}

function Barra() {
  const tema = useContext(TemaContext);
  return <div className={`barra ${tema}`}>Barra en tema {te
}
```

- **Prop drilling**: pasar un dato por niveles intermedios que no lo usan, solo para que llegue abajo.
- `createContext` crea el canal; el **proveedor** fija el valor para el subárbol; `useContext` lo lee a cualquier profundidad. Casos típicos: tema, idioma, usuario.
- No sustituye a levantar el estado: es para datos que necesita **medio árbol**, no para ahorrarse dos props.

> 📌 React 19: `<TemaContext value="…">` directamente, sin `<TemaContext.Provider>`.

`useReducer`: estado complejo con transiciones nombradas

```
import { useReducer } from "react";

function reducer(estado, accion) {
  switch (accion.tipo) {
    case "incrementar":
      return { cuenta: estado.cuenta + 1 };
    case "reiniciar":
      return { cuenta: 0 };
    default:
      return estado;
  }
}
```

```
function Contador() {
  const [estado, dispatch] = useReducer(reducer, { cuenta: 0 });
  return (
    <>
      <p>{estado.cuenta}</p>
      <button onClick={() => dispatch({ tipo: "incrementar" })}>+1
      <button onClick={() => dispatch({ tipo: "reiniciar" })}>Reir
    </>
  );
}
```

- Para estado con **varias propiedades relacionadas** o **muchas transiciones**: toda la lógica de actualización en **una función pura** (`reducer`), los componentes solo **despachan acciones**.
- Es el motor del **reto jefe** (`destapar`, `resolverTurno`, `tic`, `reiniciar`) y es el mismo patrón que `Array.reduce` de la U2.

`useMemo` y `useCallback`: memoizar con cabeza

```
// Memoiza el RESULTADO de un cálculo costoso
const valorCalculado = useMemo(
  () => calculoCostoso(datos),
  [datos]
);

// Memoiza una FUNCIÓN (misma referencia entre rende
const manejar = useCallback(
  () => hacerAlgo(id),
  [id]
);
```

- `useMemo`: guarda el resultado y solo lo recalcula si cambian las dependencias.
- `useCallback`: guarda la función; útil al pasarla a hijos optimizados para que no se repinten sin motivo.
- Regla: **no abuses**. Solo con un cálculo realmente caro o un problema de rendimiento **medido**.
- El **React Compiler** (estable 1.0, octubre de 2025) memoiza automáticamente: la tendencia es escribir **menos** memoización a mano.

> Filtrar ocho películas no es un cálculo costoso. Un `useMemo` ahí es ruido.

Hooks personalizados: reutilizar lógica con estado

```
// useReloj.js — el reloj de antes, extraído
import { useState, useEffect } from "react";

export function useReloj() {
  const [hora, setHora] = useState(new Date());

  useEffect(() => {
    const id = setInterval(() => setHora(new Date()),
    return () => clearInterval(id);
  }, []);

  return hora;
}
```

```
// Cualquier componente lo consume en una línea
function Reloj() {
  const hora = useReloj();
  return <p>{hora.toLocaleTimeString()}</p>;
}
```

- Una **función normal** que empieza por `use` y llama a otros hooks. No es una API especial.
- Comparten **lógica, no estado**: cada componente que llama a `useReloj()` tiene su propia hora.
- El prefijo `use` la somete a las reglas de los hooks (por eso existe la regla 2).

> 🎁 Bonus de la misión M3: extraer un hook reutilizable (`useFetch`, `useLocalStorage`…).

React 19: el hook `use` y las Actions

```
import { use } from "react";

function Perfil({ usuarioPromesa }) {
  const usuario = use(usuarioPromesa); // espera a la pr
  return <h1>{usuario.nombre}</h1>;
}
```

- `use` **lee el valor de una promesa** (o de un contexto) durante el render y se integra con `<Suspense>` para el estado de carga. Puede llamarse **condicionalmente**.
- ⚠️ La promesa **no se crea en el render** (`use(fetch(...))` crearía una nueva en cada render): llega cacheada, por props o del framework. Y hace falta un `<Suspense>` **por encima**.

Hooks de Actions (operaciones asíncronas como enviar un formulario):

- `useActionState`: estado resultante de una acción (datos, error, pendiente).
- `useFormStatus`: estado de envío del `<form>` padre (`pending`).
- `useOptimistic`: cambio provisional en la interfaz mientras la operación real termina.

> Solo presentación: se trabajan a fondo en la **Unidad 4**. El reto jefe tiene un reto opcional con `useActionState` (logro 🚀 «Viajero del futuro»).

Misión 3 · Videoclub Nébula 🎬

🟡 **IA como pair · ⏱️ 2–3 h** · cuaderno 3, nivel 2

- 🎯 **Objetivo:** buscador con filtros sobre datos locales, tema claro/oscuro global y `useEffect` **solo donde está justificado**.
- 🧰 **Practicas:** estado derivado (filtrado en el render), `useContext`, `useEffect` con criterio, composición.

Fases

1. **Catálogo:** `Catalogo` → `TarjetaPelicula` en rejilla (8 películas).
2. **Búsqueda derivada:** input por título + `<select>` de género. La lista visible se **calcula en el render**. Prohibido guardarla en estado o filtrar con un efecto.
3. **Tema con contexto:** `TemaContext` con `{ tema, alternarTema }`; botón 🌙/🌞 en `Cabecera`; las tarjetas leen el tema con `useContext`.
4. **Un efecto justificado:** sincronizar `document.title` y `localStorage`; leer el tema inicial con `useState(() => localStorage.getItem("tema") ?? "claro")`.

✅ Criterios

- ☐ La lista filtrada no está en ningún `useState`.
- ☐ Búsqueda y género se combinan.
- ☐ El tema llega a las tarjetas por `useContext`, sin prop drilling.
- ☐ El **único** `useEffect` es el de título/localStorage.
- ☐ Con cero resultados, mensaje amable.

```
PELICULAS.filter((p) =>
  p.titulo.toLowerCase().includes(busqueda.toLowerCase()) &&
  (genero === "todos" || p.genero === genero))
```

🚀 **Retos:** ordenación por nota o año (copia ordenada: `[...filtradas].sort(...)`) · favoritas ⭐ con filtro «solo favoritas» (¿dónde vive ese estado?).

11 · Pensar en React y buenas prácticas

El método oficial para construir cualquier interfaz, y la lista de errores que la rúbrica penaliza.

Pensar en React: el método en seis pasos

1. **Descompón la UI en componentes**: dibuja recuadros sobre el diseño. Regla: **responsabilidad única**.
2. **Organízalos en una jerarquía**: qué contiene a qué.
3. **Construye una versión estática** primero: solo props, sin estado.
4. **Identifica el estado mínimo**: de cada dato, ¿cambia con el tiempo? ¿se puede **calcular** a partir de otro? Si se puede calcular, **no es estado**.
5. **Decide dónde vive cada estado**: en el ancestro común si lo comparten varios.
6. **Añade el flujo inverso**: las funciones con las que los hijos avisan al padre.

> 💡 Error típico: **estado redundante**. Si un valor se deriva de props o de otro estado, calcúlalo en el render. Menos estado = menos errores.

Buenas prácticas y errores comunes

- ✅ Componentes **pequeños** y con un solo propósito.
- ✅ **Nunca mutes** el estado ni las props: `{...obj}`, `[...arr]`, `map`, `filter`.
- ✅ `key` **estable** en las listas (índice solo en listas estáticas).
- ✅ Limpieza en `useEffect` con temporizadores y suscripciones.
- ⚠️ No llames a `setEstado` **durante el render**: bucle infinito.
- ⚠️ No abuses de `useEffect`: ¿render? ¿manejador?
- ⚠️ Hooks solo en el nivel superior.
- ⚠️ `onClick={f}`, no `onClick={f()}`.
- ⚠️ `&&` con números: convierte a booleano.

Error común	Solución
`props.x = ...`	Usar estado; las props son inmutables
`setLista(lista.push(x))`	`setLista([...lista, x])`
`key={index}` en lista dinámica	`key={item.id}`
`onClick={borrar()}`	`onClick={() => borrar()}`
Efecto para derivar datos	Calcular durante el render
Hook dentro de un `if`	Hook en el nivel superior
`{n && <p/>}` pinta `0`	`{n > 0 && <p/>}`

> Son los hábitos que revisa el evaluador de la Arena y los defectos que buscarás en la actividad A1.

Actividad A1 · Code review al replicante 🤖

⏱ **50 min · individual o parejas · prepara la misión M3** · `ejercicios/discernment/A1-code-review-replicante/`

Una IA ha generado `Leaderboard.jsx`, el ranking de un juego. Compila, se renderiza y **a primera vista funciona**. Pero contiene **3 bugs sutiles y 2 malas prácticas**, y quien lo entregue tal cual sufrirá en la defensa.

Tu tarea: un informe tipo *pull request review* con una entrada por defecto:

1. **Dónde está** (línea o fragmento).
2. **Por qué es un defecto**, con el concepto de la U3 que lo explica (inmutabilidad, estado derivado, dependencias de efectos, keys…).
3. **El arreglo propuesto** (código).
4. **Cómo lo demostrarías en el navegador** (pista: `StrictMode` es tu amigo).

✅ Criterios de aceptación

- ☐ Las 5 entradas: 3 bugs y 2 malas prácticas.
- ☐ Cada entrada cita el **concepto** que la justifica, no solo «esto está mal».
- ☐ Tus arreglos **no introducen estado nuevo** donde bastaba derivar.
- ☐ Sabes **reproducir en vivo** al menos 2 de los defectos.

> Este es el trabajo que harás toda tu carrera: revisar código que no escribiste tú (una IA o un compañero) y decidir si pasa a producción.

ACTIVIDAD DE AULA

A1 · Puesta en común (últimos 15 min)

Dinámica

1. Se proyecta `Leaderboard.jsx`.
2. Se le pide a una IA **en vivo**: *«revisa este componente»*.
3. Comparamos su *review* con las vuestras:
   - ¿Qué cazó cada uno?
   - ¿Qué se le **escapó** a la IA?
   - ¿Qué os saltasteis **vosotros**?
4. Dos personas reproducen un defecto cada una en el navegador.

Moraleja de la sesión

> La *review* de una IA es un **punto de partida**, no un veredicto.

- La IA suele cazar lo sintáctico (la `key`, el `push`) y le cuesta más lo conceptual (estado que no debería existir).
- En la Arena, el evaluador también es una IA: **sabe** lo que sabe esta. Lo que la IA no cazó hoy es lo que te preguntarán en la defensa.
- Regla del curso: puedes usar IA para todo, pero **defiendes línea a línea** lo que entregas.

Reto jefe · Memoria Cósmica 🧠

🟢 **IA libre (declarada)** · ⏱️ **3–4 h** · cuaderno 3, nivel 3

Juego de parejas: 16 cartas boca abajo (8 parejas de emojis); destapas dos por turno y, si coinciden, quedan resueltas. Gana quien acabe en menos movimientos.

Fases

1. **Tablero estático:** `Juego` → `Tablero` → `Carta`, rejilla 4×4, `key` estable.
2. **Reducer y destape:** acción `destapar`; «boca arriba» es un dato **derivado**.
3. **Resolver el turno:** un `useEffect` observa `destapadas`; a las 2, espera **800 ms** y despacha. **La limpieza cancela el timeout.**
4. **Cronómetro y victoria:** `setInterval` → `tic` (¡con limpieza!). Victoria **derivada** con `every`.
5. **Pulido:** marcador visible, resueltas atenuadas, sin trampas.

Código de partida (`src/reto/logica.js`, en el cuaderno)

- `crearMazo()`: 16 cartas `{ id, emoji, resuelta }` barajadas.
- `estadoInicial`: `{ cartas, destapadas: [], movimientos: 0, segundos: 0 }`.
- Esqueleto del `reducer` (`destapar`, `resolverTurno`, `tic`, `reiniciar`) con dos `TODO`.

🚀 Retos

- Niveles de dificultad (4×3, 4×4, 6×4) antes de empezar.
- Récord personal en `localStorage`.
- **React 19 (opcional):** guardar la puntuación con `useActionState` y «Guardando…» mientras está pendiente (adelanto de la U4).

Reto jefe · Rúbrica de autoevaluación

Criterio	🥉 Aprobado	🥈 Notable	🥇 Sobresaliente
Estado	El juego funciona con `useReducer`	Todas las transiciones pasan por el reducer, sin `useState` paralelos	El reducer es una función pura testeable: mismas entradas, mismo estado
Inmutabilidad	No hay mutaciones directas	Uso sistemático de spread, `map` y `every`	Ninguna estructura del estado se comparte mutada entre renders
Efectos	El tablero se resuelve solo tras 800 ms	Timeout **e** intervalo con limpieza correcta	Sin dobles disparos con `StrictMode` y sin warnings de dependencias
Derivación	La victoria se detecta	Victoria y «tablero bloqueado» son derivados, no estado	Cero estados redundantes en todo el juego
Componentes	Todo en un fichero, pero funciona	`Juego`, `Tablero`, `Carta` y `Marcador` separados con props claras	`Carta` es 100 % presentacional: solo props y eventos

> Las filas **Estado**, **Efectos** y **Derivación** son las que se examinan en la defensa oral de la Arena.

Hacia la Arena · Misión M3 «Reactor Core»

Todo es un componente.

Qué se entrega: una URL de GitHub (repo o subcarpeta) con una app **React 19** creada con Vite: reconstruye (o reinventa) tu app de la misión anterior. La UI debe **reaccionar al estado**, no al revés.

Cómo se puntúa (/100)

- **30** checks automáticos: README, repo limpio, ≥ 5 commits, `package.json` con `react`, ficheros `.jsx` / `.tsx`.
- **70** rúbrica evaluada por IA: Componentización **20** · Gestión de estado **15** · Efectos **15** · Listas y condicionales **10** · Originalidad **10**.
- 🎁 Bonus: extraer un **custom hook** reutilizable (`useFetch`, `useLocalStorage`…).

La nota es provisional hasta la defensa oral. La IA genera 3 preguntas sobre **tu** código; en el ranking cuenta el mejor intento.

- README con sección **«Uso de IA»** (qué pediste, cómo lo verificaste, qué escribiste a mano) y **«Autopsia»** (las 2 decisiones más discutibles y la alternativa que descartaste).
- Estado **mínimo** y derivado; **cero** mutaciones; `key` estable; `useEffect` solo justificado y con limpieza.
- Sin componentes-dios: si un fichero pasa de 150 líneas, sospecha.

> Candidato natural: el reto jefe con CSS propio y README.

Entrena la defensa oral

Preguntas del mismo corte que las de la Arena. Se responden en 1–2 minutos **si has entendido**; de memoria son imposibles.

1. Enséñame todos los `useState` de tu aplicación. ¿Alguno se podría **calcular** a partir de los otros? Justifica por qué cada uno es estado de verdad.
2. Actualizas un objeto del estado y la pantalla no cambia. ¿Cuál es la causa más probable y cómo la arreglas? ¿Qué **compara** React exactamente?
3. Renderizas una lista con `map`. ¿Qué usaste como `key` y por qué no el índice? ¿Qué se rompería **exactamente** con el índice?
4. Enséñame un `useEffect` de tu código. ¿Por qué necesita ser un efecto y no lógica en un manejador de evento o un cálculo en el render?
5. Si tu efecto abre un intervalo, ¿qué devuelves? ¿Qué pasaría sin eso **cuando cambie una dependencia**, no solo al desmontar?
6. Dos componentes necesitan el mismo dato. ¿Dónde pusiste el estado y cómo llega el cambio desde el hijo hasta arriba?

> Las ocho preguntas con respuesta modelo están en la **autocomprobación** de la unidad, en la Arena (`/teoria`).

Tabla de logros del cuaderno 3

Logro 🏅	Cómo desbloquearlo
🧱 **Constructor**	Completar los 4 ejercicios del Nivel 1
🧠 **Cerebro de quiz** · 🔥 **Forjador** · 🎬 **Cinéfilo**	Terminar cada misión con todos los criterios marcados
🧊 **Inmutable**	Todo el cuaderno sin mutar estado ni props una sola vez
🪝 **Domador de hooks**	Todos los `useEffect` con limpieza correcta y sin warnings del linter
🏆 **Cazador de parejas**	Reto jefe funcionando de principio a fin
💎 **Perfeccionista**	Reto jefe con 🥇 en los cinco criterios de la rúbrica
🚀 **Viajero del futuro**	Completar el reto opcional con `useActionState` (adelanto de la U4)

Recursos y próxima unidad

Para profundizar

- Documentación oficial de React (19.2): https://react.dev
- Aprende React (tutorial oficial): https://react.dev/learn
- Pensar en React: https://react.dev/learn/thinking-in-react
- No siempre necesitas un efecto: https://react.dev/learn/you-might-not-need-an-effect
- Reglas de los hooks: https://react.dev/reference/rules/rules-of-hooks
- Novedades de React 19 y React Compiler: https://react.dev/blog · https://react.dev/learn/react-compiler
- Vite: https://vite.dev

Unidad 4 — Formularios y peticiones HTTP

- Formularios controlados y no controlados; **React Hook Form** con `register` / `handleSubmit`.
- Validación con **Zod** como única fuente de verdad; formularios accesibles.
- Repaso de HTTP, `fetch` a fondo y **el problema de `fetch` dentro de `useEffect`**.
- **TanStack Query v5**: `useQuery`, `useMutation`, invalidación.
- Actions de React 19, CORS, seguridad en el cliente.
- Misión M4 «Full Contact Forms»: *valida o muere*.